

操作系统第七章作业

7-12

已知主存有256KB容量，其中OS占用低址20KB，可以有这样的一个程序序列。

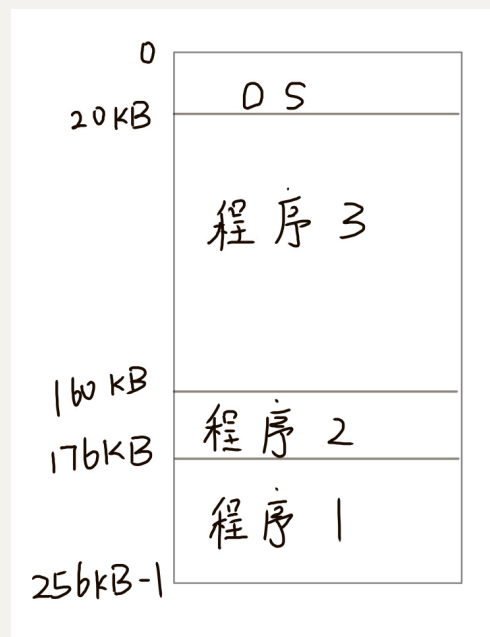
程序1要求 80KB；程序2要求16KB；程序3要求140KB。

程序1完成；程序3完成。

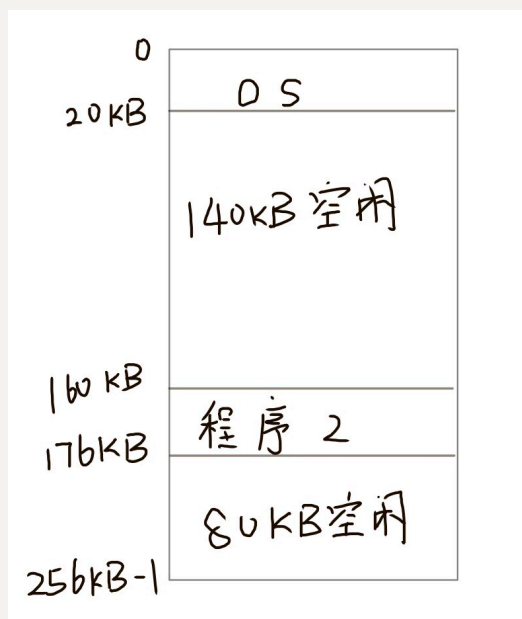
程序4要求 80KB；程序5要求120KB。

试分别用首次适应算法和最佳适应算法分别处理上述程序序列 (在存储分配时，从空白区高址处分割作为已分配区)，并完成以下各步骤。

(1) 画出程序1、2、3进入主存后主存的分配情况。



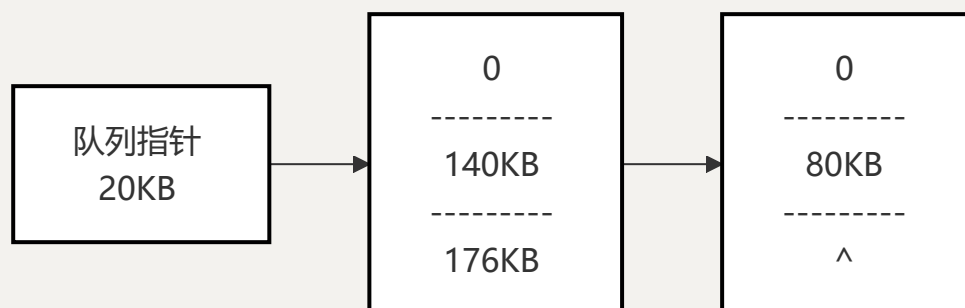
(2) 画出程序1、3完成后主存分配情况。



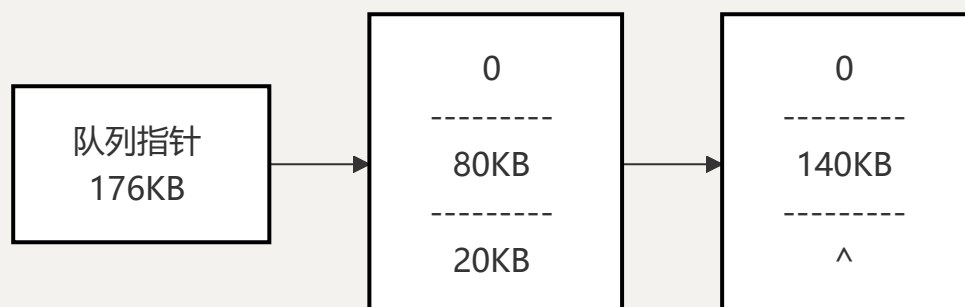
(3) 试用上述两种算法中画出程序1、3完成后的空闲区队列结构 (要求画出分区描述器信息，假定分区描述器所需占用的字节数已包含在程序所要求的主存容量中)。

使用 Mermaid 代码绘制图片

首次适应算法 (**First Fit, FF**): 空闲分区链通常按地址递增顺序排列。



最佳适应算法 (**Best Fit, BF**): 空闲分区链通常按容量(大小)递增顺序排列。



(4) 哪种算法对该程序序列而言是适合的？简要说明分配过程。

应该使用最佳适应算法

1. 首次适应算法 (First Fit) - 分配失败

- 程序4 (80KB)：优先选首个满足的空闲区A (140KB)，高址分割 (80~160KB)，剩余A: 60KB (20~80KB)
- 程序5 (120KB)：检查空闲区A (60KB)、B (80KB)，均不满足，分配失败

2. 最佳适应算法 (Best Fit) - 分配成功

- 程序4 (80KB)：选最小适配空闲区B (80KB)，完全占用，无剩余
- 程序5 (120KB)：选唯一适配的空闲区A (140KB)，高址分割 (40~160KB)，剩余A: 20KB (20~40KB)，分配成功

最佳适应算法可完成全部分配，首次适应算法因碎片化无法满足程序5需求。

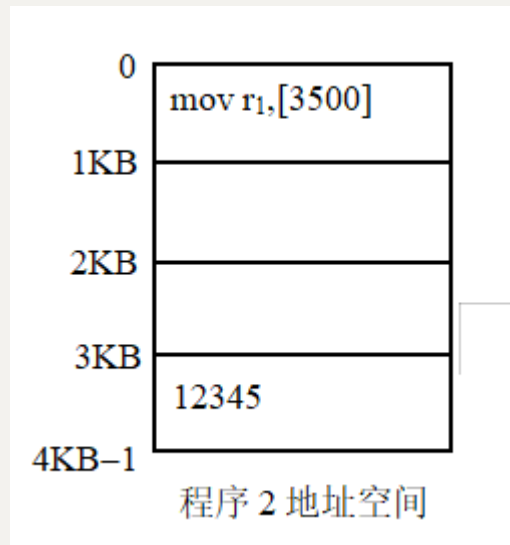
7-14

已知主存容量为64K字节，某一程序A的地址空间如图7.46所示，它的4个页面 (页面大小为1KB字节) 0、1、2、3被分配到主存的2、4、6、7块中。

(1) 画出A的页面映像表；

页号 (PAGE NO.)	块号 (BLOCK NO.)
0	2
1	4
2	6
3	7

(2) 当200号单元处有一条指令“mov r1, [3500]”执行时，如何进行正确的地址变换，以使3500处的内容12345装入r1中？



地址转换过程

1. 计算页号与页内偏移

页面大小 1KB=1024 字节，逻辑地址 3500:

页号 $P = 3500 \div 1024 = 3$ ，页内偏移 $W = 3500 \% 1024 = 428$ 。

2. 查页表得物理块号

页号 3 对应块号 $f = 7$ 。

3. 计算物理地址

物理地址 $= 7 \times 1024 + 428 = 7596$ 。

所以物理地址为 7596

7-21

在请求分页系统中，某程序 A 有 10 个页面，系统为其分配了 3 个主存块。设该程序第 0 页已装入主存，进程运行时访问页面的轨迹是 0 1 3 0 5 2 0，试用页号栈的方法回答如下问题。

(1) 在先进先出页面置换算法下，缺页中断次数是多少？要求用图画出每一次页面置换前后的情况。

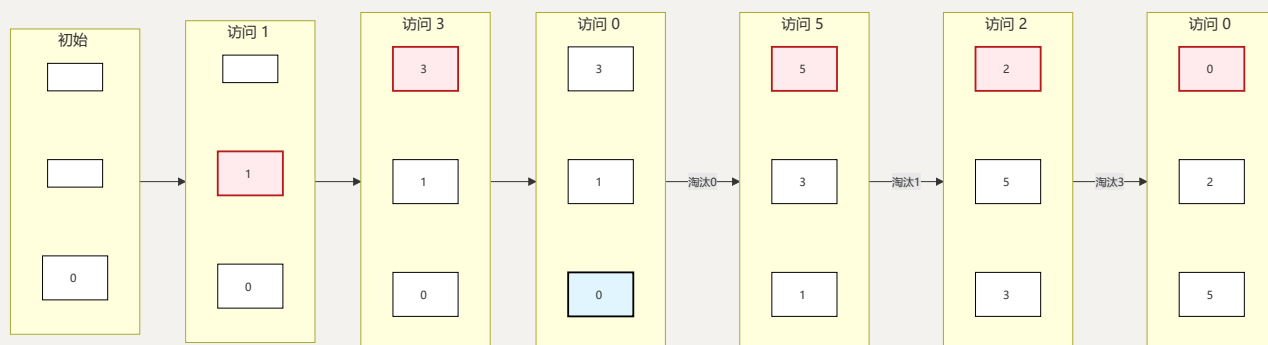
- 什么是先进先出淘汰算法：
 - 总是选择在主存中居留时间最长 (即最早进入主存) 的一页淘汰。
- 先进先出淘汰算法的实现

- 建立一个页面进入主存的先后次序表；
- 建立一个替换指针，指向最早进入主存的页面；
- 当需要置换一页时，选择替换指向的那一页，然后调整替换指针的内容。

1. 初始状态：内存中有页面 **0**。
2. 访问 **1**：缺页。1 压入栈顶。栈：2->1, 1->0。
3. 访问 **3**：缺页。3 压入栈顶。栈满：3->2, 2->1, 1->0。0 在栈底（最老）。
4. 访问 **0**：命中。栈内已有 0，FIFO 不调整顺序。栈维持：3, 1, 0。
5. 访问 **5**：缺页。淘汰栈底的 0，5 压入栈顶。栈：5, 3, 1。1 变栈底。
6. 访问 **2**：缺页。淘汰栈底的 1，2 压入栈顶。栈：2, 5, 3。3 变栈底。
7. 访问 **0**：缺页。淘汰栈底的 3，0 压入栈顶。栈：0, 2, 5。

缺页次数：5次

使用 **Mermaid** 代码绘制图片



(2) 若采用最久未使用置换算法，回答上述同样问题。

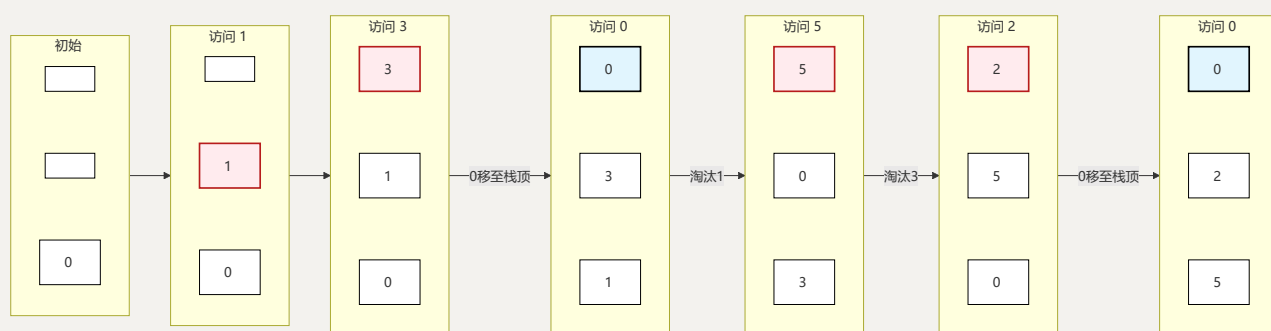
- 什么是最久未使用淘汰算法
 - 总是选择最长时间未被使用的那一页淘汰。
- 最久未使用淘汰算法的实现
 - 用引用位考察页面的使用情况；
 - 当访问页面时，将引用位置1，并计时；
 - 当要淘汰一页时，选择时间最长的一页淘汰。

1. 初始状态：栈：0。

2. 访问 **1**：缺页。1 入栈顶。栈：**1, 0**。
3. 访问 **3**：缺页。3 入栈顶。栈：**3, 1, 0**。（此时0在栈底，为最久未使用）。
4. 访问 **0**：命中。将 0 从栈底抽出，移至栈顶。栈：**0, 3, 1**。（此时1沉底）。
5. 访问 **5**：缺页。栈满，淘汰栈底的 **1**。5 入栈顶。栈：**5, 0, 3**。
6. 访问 **2**：缺页。栈满，淘汰栈底的 **3**。2 入栈顶。栈：**2, 5, 0**。
7. 访问 **0**：命中。将 0 从栈底抽出，移至栈顶。栈：**0, 2, 5**。

缺页次数：**4**次

使用 **Mermaid** 代码绘制图片



7-32

某页式存储管理系统实现时结合简化的段式管理，虚拟地址长度为24位，其中23~22两位表示段类型：01、10、11分别代表代码段、数据段和栈段，00非法；主存块和页面大小为2 K。现有一进程P，代码段分别占用4个主存块0xA、0x8、0x5、0xF，数据段分别占用3个主存块0xB、0x7、0xD，栈段分别占用两个主存块0x1、0x6，回答以下问题：

（1）该页式存储管理系统中，进程的代码段空间以及整个虚拟地址空间最大是多少？

代码段空间最大值：虚拟地址总长24位，高2位（23~22位）用于区分段类型。这意味着剩余的 22位（21~0位）用于表示段内偏移（即段内的寻址空间）。因此，单个段（如代码段）的最大长度为：

$$2^{22} \text{ Byte} = 4 \text{ MB}$$

整个虚拟地址空间最大值：整个虚拟地址长度为 24位。因此，整个虚拟地址空间的大小为： $2^{24} \text{ Byte} = 16 \text{ MB}$

段号只有 01、10、11 合法，00 非法，所以进程可用的虚拟空间实际为：12 MB

(2) 画出进程P主存中段表、页表结构，其中段表包含段的起始虚拟地址、页表指针。

段表结构（假设段表在主存中，以表格表示）：

段类型	段名	起始虚拟地址	页表指针
01	代码段	0x400000	
10	数据段	0x800000	
11	栈段	0xC00000	

页表结构（按给定主存块顺序假设页号从0开始连续分配）：

- 代码段页表（4个页，指向主存块0xA、0x8、0x5、0xF）：

页号	物理块号
0	0xA
1	0x8
2	0x5
3	0xF

- 数据段页表（3个页，指向主存块0xB、0x7、0xD）：

页号	物理块号
0	0xB
1	0x7
2	0xD

- 栈段页表（2个页，指向主存块0x1、0x6）：

页号	物理块号
0	0x1
1	0x6

(3) 计算出进程P中逻辑地址8006ADH的物理地址，给出计算过程。

[段号(2) | 页号(11) | 偏移(11)]
[10 | 00000000000 | 11010101101]

1. 确定段类型：位23~22为10（二进制），对应数据段。

2. 计算页号和页内偏移：

- 页面大小为2 KB = 2^{11} 字节，页内偏移为低11位（位10~0）：二进制11010101101 = 0x6AD（十进制1709）。
- 页号为高11位（位21~11）：二进制00000000000 = 0。

3. 查数据段页表：数据段页号0对应的物理块号为0xB（十进制11）。

4. 计算物理地址：物理地址 = (物理块号 × 页面大小) + 页内偏移, 每块大小 = 2K = 0x800 字节。

结果为 $(0xB \times 0x800) + 0x6AD = 0x5800 + 0x6AD = 0x5EAD$ 。

最终结果：物理地址为 **5EAD H**。